

# 2.1 Python for Edge AI Engineers

## Edge AI Engineering Bootcamp | Analog Data

### Why This Matters

You'll be running multiple AI/embedded projects on the same Raspberry Pi — one might need `tensorflow==2.13`, another might need `tensorflow==2.15`. If you install everything into one shared system Python, these projects will eventually collide and break each other. This module is about keeping every project in its **own isolated box**, and making sure anyone (including future-you) can rebuild that exact box on another machine.

### 1. The Problem — Why Isolation Matters

WITHOUT isolation (everything shared):

System Python

```
numpy 1.21
tensorflow 2.13
opencv 4.5
```

▲  
|  
Project A needs  
tensorflow 2.13  
→ works fine

▲  
|  
Project B needs  
tensorflow 2.15  
→ BREAKS Project A  
when installed!

If two projects need **different versions** of the same package, and everything is installed into one shared Python, installing the second project's requirements can silently break the first one. This is exactly the kind of bug that seems to make no sense — "my code hasn't changed, why did it stop working?" — until you realize a completely unrelated `pip install` from another project touched the same shared environment.

**The fix:** give every project its own private copy of Python and its own private set of packages.

WITH isolation (each project boxed separately):

Project A's venv

```
numpy 1.21
tensorflow 2.13
opencv 4.5
```

Project B's venv

```
numpy 1.26
tensorflow 2.15
opencv 4.9
```

Completely separate — installing/upgrading  
one has ZERO effect on the other

## 2. venv — Python's Built-In Isolation Tool

`venv` creates a self-contained folder holding its own Python interpreter and its own `site-packages` (where installed libraries live). It comes built into Python — no extra installation needed.

### Creating and using a virtual environment

```
# Create a new environment named "myenv" in the current folder
python3 -m venv myenv

# Activate it (Linux/Mac/Raspberry Pi OS)
source myenv/bin/activate

# Your terminal prompt changes to show you're inside it:
(myenv) pi@raspberrypi:~ $

# Now any pip install goes ONLY into this environment
pip install numpy opencv-python

# When you're done working, leave the environment
deactivate
```

#### What actually happens on disk:

```
myenv/
├── bin/
│   ├── python3      ← a private copy/link of the interpreter
│   ├── pip          ← pip that installs INTO this env only
│   └── activate      ← the script you "source" to enter the env
├── lib/
│   └── python3.11/
│       └── site-packages/ ← all your pip-installed packages live here,
                           completely separate from the system
└── pyvenv.cfg
```

**Key idea for students:** activating an environment doesn't move or copy your project files — it just changes which `python` and `pip` your terminal points to for that session. Deactivating snaps you right back to the system Python.

### Checking which Python you're actually using

```
which python3      # shows the FULL PATH currently active
```

```
# Outside any venv:
/usr/bin/python3
```

```
# Inside an activated venv:  
/home/pi/myproject/myenv/bin/python3
```

This is the single most useful debugging command when something feels "wrong" — always confirm which Python you're actually pointed at before assuming a package is "missing."

## Confirming It From *Inside* Python Itself

which `python3` checks from the terminal's point of view. But you can ask the exact same question **from inside a running Python script**, which is often even more reliable — especially when you're debugging inside an IDE, a Jupyter notebook, or a script launched by something else (like a `systemd` service) where you're not sure what environment it actually picked up.

```
import sys  
  
print(sys.executable)
```

Run it outside any venv:

```
python3 check_env.py
```

```
/usr/bin/python3
```

Now activate your venv and run the exact same script:

```
source venv/bin/activate  
python3 check_env.py
```

```
/home/pi/myproject/venv/bin/python3
```

**Same script, same code — different output — because `sys.executable` always tells the truth about which actual Python binary is running your code, right now, no guessing.**

You can go one step further and print exactly where Python is looking for installed packages:

```
import sys  
  
print("Python executable:", sys.executable)  
print("Python version:", sys.version)  
print("\nWhere Python looks for packages:")  
for path in sys.path:  
    print(" -", path)
```

**Why this matters more than it sounds:** a huge number of "but I installed it, why does it say `ModuleNotFoundError`?" bugs are actually just this — the student installed a package into one environment, but ran their script using a *different* Python. `sys.executable` ends the guessing in two lines of code, and it works everywhere `which` might not (inside notebooks, IDEs, cron jobs, services).

## One venv per project — the standard convention

```
mkdir edge_ai_project
cd edge_ai_project
python3 -m venv venv          # convention: name it "venv" inside the project
                              folder
source venv/bin/activate
pip install -r requirements.txt
```

**Note:** on Raspberry Pi OS Bookworm, `venv` is often the *required* way to install packages cleanly, since plain system-wide `pip install` is blocked by default (covered in Module 1.3 — `--break-system-packages`).

### 3. `pyenv` — Managing Multiple *Python Versions*

`venv` isolates **packages** for a single Python version already installed on your system. But what if two projects need **different Python versions entirely** — one needs Python 3.9, another needs Python 3.11? That's a different problem, and that's what `pyenv` solves.



#### Basic `pyenv` commands

```
# Install pyenv (typical method on Raspberry Pi OS / Debian-based systems)
curl https://pyenv.run | bash

# List Python versions available to install
pyenv install --list

# Install a specific Python version (compiles from source — can take a while on a Pi!)
pyenv install 3.11.6

# See which versions are currently installed
pyenv versions
```

```
# Set a Python version for ONE specific project folder
cd myproject
pyenv local 3.11.6      # creates a .python-version file in this folder

# Set a Python version globally (system-wide default)
pyenv global 3.11.6
```

### Combining pyenv + venv (the professional-grade workflow):

```
cd myproject
pyenv local 3.11.6      # this project uses Python 3.11.6 specifically
python3 -m venv venv    # create a venv USING that exact Python version
source venv/bin/activate
pip install -r requirements.txt
```

**Important practical note for Raspberry Pi:** `pyenv install` **compiles Python from source**, which is slow on a Pi's ARM CPU (can take 20–40+ minutes for one version). For most bootcamp projects, the Python version that ships with Raspberry Pi OS is sufficient, and plain `venv` is all you need. `pyenv` becomes valuable when you're specifically working across projects that genuinely require different Python versions — know it exists, but don't reach for it unnecessarily on lower-powered hardware.

## 4. `uv` — The Fast, Modern Replacement for `pip` + `venv`

`uv` is a newer tool (built by the makers of the popular `ruff` linter) that has rapidly become one of the most talked-about tools in the Python world. It replaces `pip`, `venv`, and even parts of `pyenv` with a **single, much faster tool**, written in Rust instead of Python.

**Why it's become so popular so quickly:**

Installing the same packages...

|                   |                        |              |
|-------------------|------------------------|--------------|
| <code>pip:</code> | <div><div></div></div> | ~25 seconds  |
| <code>uv:</code>  | <div><div></div></div> | ~1–2 seconds |

`uv` is often **10–100x faster** than plain `pip`, because it's written in a compiled systems language (Rust) and uses much smarter caching and parallel downloads. On a Raspberry Pi — where CPU power is limited compared to a laptop — this speed difference is even more noticeable, especially when installing heavy packages like `numpy` or `opencv-python`.

### Installing `uv`

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

### Creating a venv with `uv` (much faster than `python3 -m venv`)

```
uv venv      # creates a .venv folder, auto-detects Python version
source .venv/bin/activate
```

## Installing packages with uv

```
uv pip install numpy opencv-python      # same idea as pip, dramatically faster
```

## Generating requirements.txt with uv

```
uv pip freeze > requirements.txt        # works exactly like pip freeze
```

## Installing from requirements.txt with uv

```
uv pip install -r requirements.txt      # rebuilds the environment, much faster
```

## The newer, all-in-one workflow (uv's own project system)

uv also has its own modern project management system, similar to tools like poetry , but faster:

```
uv init myproject          # creates a new project folder with proper structure
cd myproject
uv add numpy opencv-python  # adds packages AND updates the project's dependency
                             # file automatically
uv run main.py              # runs your script using the correct project
                             # environment automatically
```

How uv compares to what we've already covered:

| Task                        | Traditional Tool                | uv Equivalent                      |
|-----------------------------|---------------------------------|------------------------------------|
| Create isolated environment | python3 -m venv venv            | uv venv                            |
| Install a package           | pip install numpy               | uv pip install numpy               |
| Freeze dependencies         | pip freeze > requirements.txt   | uv pip freeze > requirements.txt   |
| Install from file           | pip install -r requirements.txt | uv pip install -r requirements.txt |
| Manage Python versions      | pyenv install 3.11.6            | uv python install 3.11.6           |

**Should you replace everything you just learned with uv?** No — understanding venv , pip , and requirements.txt first is important, because uv is largely a **faster drop-in replacement** for the exact same underlying concepts. Every environment, package list, and requirements.txt file you build with uv works the same way conceptually as what pip / venv produce. Once you're comfortable with why isolation and reproducibility matter, switching to uv for speed is simply a practical upgrade — not a different way of thinking.

**Practical recommendation for this bootcamp:** on a Raspberry Pi, where installing packages like tensorflow or opencv-python can otherwise take a long time, uv is genuinely worth adopting early — the time savings compound across every project you build for the rest of this course.

## 5. requirements.txt — Recording Exactly What Your Project Needs

A `requirements.txt` file is a plain text list of every package (and ideally, exact version) your project depends on. It's what makes your project **reproducible** — anyone (including you, six months from now, on a brand new Pi) can rebuild the exact same environment with one command.

### What it looks like

```
numpy==1.26.4
opencv-python==4.9.0.80
tensorflow==2.15.0
picamera2==0.3.17
gpiozero==2.0.1
```

The double-equals operator (written as two "=" characters together) pins an **exact version** — this is important. Without it, `pip install numpy` might install a different (newer) version on a different machine, and behavior can subtly change or break.

### pip freeze — Generating the File Automatically

Once you've installed everything your project needs inside its venv, you don't write `requirements.txt` by hand — you generate it directly from what's actually installed:

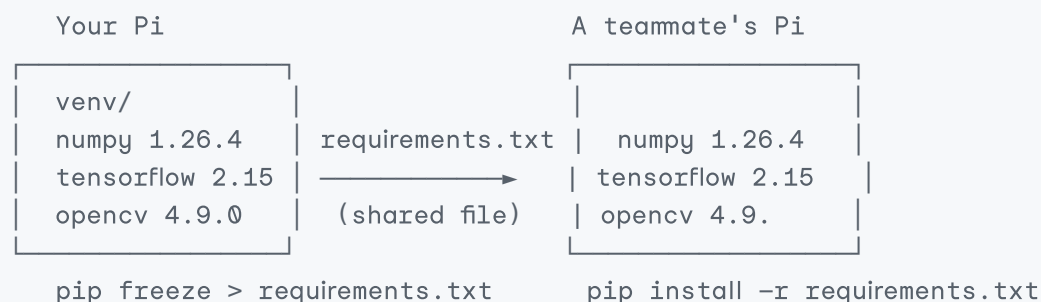
```
source venv/bin/activate
pip freeze > requirements.txt
```

`pip freeze` prints every installed package **with its exact currently-installed version** — capturing the precise state of your working environment, not just what you remember installing.

### Rebuilding the environment elsewhere (the whole point)

```
# On a brand new Pi, or a teammate's machine:
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

This single command reproduces the **exact same set of package versions**, avoiding the classic "works on my machine" problem.



## A Common Mistake to Watch For

```
pip freeze > requirements.txt # run this WHILE your venv is activated!
```

If you accidentally run `pip freeze` **without** activating your venv first, it captures your **system-wide** packages instead of your project's isolated ones — producing a bloated, incorrect `requirements.txt` full of unrelated packages. Always check `which python3` before running `pip freeze` to confirm you're inside the right environment.

---

## Quick Recap

1. Without isolation, projects sharing one Python environment can silently break each other when package versions conflict.
  2. `venv` creates an isolated folder with its own Python + packages — one per project is the standard convention. Use `which python3` (in the terminal) or `sys.executable` (inside your code) to always confirm which Python you're actually using.
  3. `pyenv` manages multiple *Python versions* themselves (not just packages) — useful when projects genuinely need different Python versions, but slow to install from source on a Pi.
  4. `uv` is a modern, Rust-based replacement for `pip` + `venv` (and parts of `pyenv`) that's dramatically faster — same underlying concepts, much better speed, especially valuable on a Pi's limited CPU.
  5. `requirements.txt` is your project's exact "ingredient list" — always generate it with `pip freeze` (or `uv pip freeze`) while your venv is active.
  6. The full reproducible workflow: create venv → activate → install packages → freeze to `requirements.txt` → share the file → anyone rebuilds it with `pip install -r requirements.txt` (or the faster `uv` equivalent).
- 

## Self-Check Questions

1. Why can installing a package for Project B accidentally break Project A, if both share the same Python environment?
2. What's the actual difference between what `venv` isolates and what `pyenv` isolates?
3. What command would you run in the terminal, and what two lines of Python code would you write, to check exactly which Python interpreter is currently active?
4. Why must you activate your venv *before* running `pip freeze > requirements.txt`? What goes wrong if you forget?
5. In `requirements.txt`, what does the double-equals sign in `numpy==1.26.4` actually control, and why does it matter for reproducibility?
6. What is `uv`, and why might it matter more on a Raspberry Pi than on a powerful laptop?



7. You're handed a new Raspberry Pi and a project's `requirements.txt` file. Write the exact sequence of commands to get the project running with the correct isolated environment — first using `pip / venv`, then again using `uv`.